

后端部署

本手册用于部署 PostgreSQL、MQTT、Service API 和 Admin UI。目标宿主机使用一份静态 Compose 文件和一份宿主机持有的环境文件；发布工作站负责准备文件和镜像，目标宿主机只负责拉取和运行镜像。

唯一运行定义是 `apps/service-api/docker-compose.yml`。应用镜像必须填写不可变 digest 引用，例如 `ghcr.io/ykdz/vem-service-api@sha256:<64-hex>`，不能使用 `latest` 或仅使用可变 tag。

准备文件

在发布工作站执行以下命令，将 Compose 文件安装到目标宿主机的固定位置。环境文件由目标宿主机单独保存。

```
ssh <host> 'sudo install -d -m 0755 /opt/vem-backend/apps/service-api /etc/vem'
scp apps/service-api/docker-compose.yml <host>:/tmp/vem-backend-docker-compose.yml
ssh <host> 'sudo install -m 0644 /tmp/vem-backend-docker-compose.yml /opt/vem-backend/apps/service-api/docker-compose.yml && rm /tmp/vem-backend-docker-compose.yml'
```

在宿主机设置路径，并创建仅 root 可读的 `/etc/vem/backend.env`：

```
export COMPOSE_FILE=/opt/vem-backend/apps/service-api/docker-compose.yml
export ENV_FILE=/etc/vem/backend.env
sudo install -m 0600 /dev/null "$ENV_FILE"
sudoedit "$ENV_FILE"
```

环境文件必须至少包含下列 Compose 要求的值。`<...>` 表示发布时填写的实际值，不要写入仓库或公开日志。

```
POSTGRES_PASSWORD=<postgres-password>
MQTT_USERNAME=<mqtt-username>
MQTT_PASSWORD=<mqtt-password>
SERVICE_API_IMAGE=ghcr.io/ykdz/vem-service-api@sha256:<64-hex>
ADMIN_UI_IMAGE=ghcr.io/ykdz/vem-admin-ui@sha256:<64-hex>
JWT_SECRET=<jwt-secret>
JWT_REFRESH_SECRET=<jwt-refresh-secret>
BOOTSTRAP_ADMIN_PASSWORD=<bootstrap-admin-password>
MACHINE_JWT_SECRET=<machine-jwt-secret>
MACHINE_CREDENTIAL_ENCRYPTION_KEY=<64-hex-key>
MACHINE_CLAIM_LOOKUP_HMAC_KEY=<claim-lookup-hmac-key>
MACHINE_API_BASE_URL=http://<public-host>:26849/api
MACHINE_MQTT_URL=mqtt://<public-host>:1883
```

```
PAYMENT_WEBHOOK_BASE_URL=http://<public-host>:26849
PAYMENT_CONFIG_ENCRYPTION_KEY=<64-hex-key>
```

需要沿用既有数据时，再按切换前检查记录填写以下卷名。它们对应静态 Compose 中的 PostgreSQL、MQTT 和受管媒体卷；迁移既有环境时，先确认这些卷名指向原有数据。

```
POSTGRES_DATA_SOURCE=<existing-postgres-volume>
MQTT_DATA_SOURCE=<existing-mqtt-volume>
SERVICE_API_MEDIA_VOLUME_NAME=<existing-media-volume>
```

`MACHINE_API_BASE_URL` 和 `MACHINE_MQTT_URL` 必须填写为现场机器可访问的地址，VPS 迁移时不要保留容器内地址或 `localhost`。`PAYMENT_WEBHOOK_BASE_URL` 推荐直接填写公网 origin（例如 `http://118.25.104.160:26849` 或 `https://pay.example.com`）；只有在外反向代理已将 webhook 基址固定到复数路径时，才填写 `https://pay.example.com/api/payments/webhooks`，不要写成单数 `/api/payments/webhook`，也不要手工重复拼接 provider 路径。正式公网部署优先使用 HTTPS；受控验收主机可以先使用当前可访问的 HTTP 地址，避免证书和域名阻塞业务验收。

端口如需保持现有对外地址，也在同一文件设置 `SERVICE_API_PORT`、`ADMIN_UI_PORT` 和 `MQTT_PORT`。PostgreSQL 在该 Compose 形态下默认仅供容器内部访问，不发布宿主机 `5432`；未设置前述端口时 Compose 使用其文件中定义的默认值。

校验与启动

发布前可在有 Docker 的工作站执行 Compose 冒烟测试，确认静态 Compose 文件和镜像入口仍可启动：

```
node scripts/backend-compose-smoke.ts \
  --service-api-image ghcr.io/ykdz/vem-service-api@sha256:<64-hex> \
  --admin-ui-image ghcr.io/ykdz/vem-admin-ui@sha256:<64-hex>
```

先只渲染配置。该命令是目标宿主机的最小可验证接口，失败时补齐环境文件而不要改写 Compose 文件：

```
sudo docker compose --env-file "$ENV_FILE" -f "$COMPOSE_FILE" config
```

确认渲染结果中的两个应用镜像都保留 `@sha256:` 后拉取并启动：

```
sudo docker compose --env-file "$ENV_FILE" -f "$COMPOSE_FILE" pull
sudo docker compose --env-file "$ENV_FILE" -f "$COMPOSE_FILE" up -d --wait --wait-timeout 240
sudo docker compose --env-file "$ENV_FILE" -f "$COMPOSE_FILE" ps
```

Compose 依次检查 PostgreSQL `pg_isready`、带认证的 MQTT 发布、Service API `/api/health` 和 Admin UI 首页。再通过 Admin UI 代理检查后端连通性：

```
ADMIN_CONTAINER="$(sudo docker compose --env-file "$ENV_FILE" -f "$COMPOSE_FILE" ps -q admin-ui)"  
sudo docker exec "$ADMIN_CONTAINER" wget -qO- http://127.0.0.1/api/health
```

成功响应中的 `data.database` 应为 `ok`，`data.mqtt` 应为 `connected`。

日志与故障检查

```
sudo docker compose --env-file "$ENV_FILE" -f "$COMPOSE_FILE" logs --tail=200 postgres mqtt  
service-api admin-ui  
sudo docker compose --env-file "$ENV_FILE" -f "$COMPOSE_FILE" logs --tail=200 service-api  
sudo docker compose --env-file "$ENV_FILE" -f "$COMPOSE_FILE" ps
```

Service API 未就绪时先检查数据库迁移和数据库连接；MQTT 显示未连接时先检查账号密码和容器网络；Admin UI 能打开但代理健康失败时检查 `API_URL` 与 Service API 健康状态。部署失败时优先检查配置和服务日志，继续使用 `digest` 固定的预构建镜像。